Tabla de interacciones y justificación

ID	Interacción y mensaje	Tipo, protocolo y middleware	Justificación
I1	Cliente → Reservas. Buscar vuelos, retener, pagar y consultar estado.	REST sobre HTTPS, síncrona.	**R1, R4.** El cliente necesita ver la disponibilidad al buscar. Al iniciar el pago recibe el estado del intento y puede consultarlo mientras la pasarela procesa el cobro.
I2	Agencia → Reservas. Solicitar una reserva.	REST sobre HTTPS, síncrona.	**R2, R3.** La agencia usa el mismo servicio de Reservas que el cliente. Ambos canales pasan por Inventario para retener y confirmar asientos.
I3	Reservas → Inventario. Consultar, retener y confirmar asiento.	RPC síncrona con gRPC sobre HTTP/2.	**R1–R3.** Reservas espera la decisión de Inventario antes de informar que un asiento quedó retenido o confirmado. El objetivo de 1.200 consultas por segundo y p95 menor a 300 ms [1] debe comprobarse desde la solicitud del cliente hasta la respuesta.
I4	Inventario → BD. Consultar y actualizar el estado del asiento.	SQL sobre TCP, síncrona, con controlador PostgreSQL.	**R2, R3.** La retención vence a los 10 minutos. Al confirmar, una transacción breve comprueba que el asiento siga retenido por esa reserva y que la retención esté vigente. Si compiten dos solicitudes, solo una puede ocuparlo [2, sec. 13.3.2].
I5	Reservas → BD. Guardar la reserva y su estado.	SQL sobre TCP, síncrona, con controlador PostgreSQL.	**R3, R4.** Guardar el estado permite responder cuando el cliente consulta qué pasó con su compra. Reservas registra la confirmación después de que Inventario acepta el cambio del asiento.
I6	Reservas → Pagos. `IniciarPago` con reserva, monto y clave.	RPC síncrona con gRPC sobre HTTP/2. Objetivo de respuesta de hasta 2 s [3].	**R4.** Pagos devuelve el estado inicial del intento una vez registrado. La llamada puede terminar mientras el cobro sigue en curso.
I7	Pagos → Pasarela. Cobrar con una clave idempotente.	REST sobre HTTPS, síncrona desde Pagos. Tiempo límite de 30 s.	**R4, F1.** Pagos espera la respuesta de la pasarela fuera de la solicitud del cliente. Si necesita reintentar, reutiliza la clave del mismo intento según el contrato de la pasarela.
I8	Pagos → RabbitMQ. Publicar `pago.aprobado` o `pago.rechazado`.	Mensajería asíncrona con AMQP y RabbitMQ.	**R4.** El resultado se publica cuando termina el cobro. La solicitud inicial del cliente puede haber terminado para entonces.
I9	RabbitMQ → Reservas. Entregar el resultado del pago.	Mensajería asíncrona con AMQP y RabbitMQ.	**R3, R4.** Al recibir un pago aprobado, Reservas solicita la confirmación del asiento por I3. Si RabbitMQ vuelve a entregar el evento, Reservas reconoce el intento y evita procesarlo dos veces [4].
I10	Reservas → RabbitMQ. Publicar `reserva.confirmada` y `pase.emitido`.	Mensajería asíncrona con AMQP y RabbitMQ.	**R3, R5.** Los eventos salen después de guardar la confirmación. A partir de ellos pueden avanzar el aviso al pasajero y la actualización del manifiesto.
I11	RabbitMQ → Notificaciones. Entregar los datos para el aviso y el pase.	Mensajería asíncrona con AMQP y RabbitMQ.	**R5, F2.** Una cola durable conserva el trabajo pendiente si Notificaciones se cae. El objetivo se mide hasta que el aviso llega al pasajero, en menos de un minuto después del pago.

ID	Interacción y mensaje	Tipo, protocolo y middleware	Justificación
I12	Notificaciones → Proveedor de correo y SMS. Enviar el aviso y el pase.	Envío en segundo plano mediante SMTP o una API HTTPS.	**R5.** El envío ocurre después de responder al cliente. Notificaciones puede esperar la respuesta del proveedor en su propio proceso y reintentar si falla.
I13	RabbitMQ → Embarque. Entregar `pase.emitido` para actualizar el manifiesto.	Mensajería asíncrona con AMQP y RabbitMQ.	**R6, F3.** El servidor guarda los pases autorizados en el aeropuerto antes de que los necesiten los lectores.
I14	Embarque → RabbitMQ. Publicar `embarque.registrado`.	Mensajería asíncrona con AMQP y RabbitMQ. Almacenamiento local y envío posterior.	**R6, F3.** El servidor registra el uso del pase en el aeropuerto y lo comunica al centro de datos cuando hay conexión. La respuesta al lector puede salir antes.
I15	Lectores de puerta → Embarque. `RES|VUELO|ASIENTO` → válido o inválido.	Sockets TCP, síncrona en la red local [5, sec. 2.2].	**R6.** La puerta necesita una respuesta para decidir si permite el paso. El mensaje tiene formato fijo y se valida localmente. La meta es responder en menos de un segundo con 40 lectores conectados.
I16	Quioscos → Embarque. Validar pase o hacer *check-in*.	Sockets TCP, síncrona en la red local.	**R6, F3.** El quiosco también necesita un resultado inmediato. Durante un corte puede validar pases que ya figuren en el manifiesto local.

Referencias

[1] G. C. Guerrero Ulloa, «Rúbrica GA-TL-01-FORM,» Material docente, UTEQ, periodo 2026–2027, Anexo A, escenario 1, pp. 11–12. Archivo facilitado por el docente: Detalles..pdf. dirección: https://github.com/iMouraad/APPD_MPZ/blob/main/docs/Detalles..pdf

[2] PostgreSQL Global Development Group. «Explicit Locking.» Documentación oficial, sección 13.3; bloqueos de fila en la sección 13.3.2, visitado 10 de nov. de 2026. dirección: https://www.postgresql.org/docs/current/explicit-locking.html

[3] gRPC Authors. «Deadlines.» Documentación oficial, visitado 10 de nov. de 2026. dirección: https://grpc.io/docs/guides/deadlines/

[4] RabbitMQ. «Consumer Acknowledgements and Publisher Confirms.» Documentación oficial, visitado 10 de nov. de 2026. dirección: https://www.rabbitmq.com/docs/confirms

[5] W. Eddy, «Transmission Control Protocol (TCP),» RFC Editor, RFC 9293, ago. de 2022, W. Eddy, editor. Internet Standard, STD 7. Flujo de bytes en la sección 2.2, doi: 10.17487/RFC9293. visitado 10 de nov. de 2026. dirección: https://www.rfc-editor.org/rfc/rfc9293.html